

Module 1

Introduction

Data is one of the most valuable resources in today's digital world. Every organization such as banks, hospitals, educational institutions, social media companies, and e-commerce platforms continuously generates huge amounts of data.

Earlier, traditional databases were sufficient to store and process data. But now, because of:

- Social media,
- Smartphones,
- Sensors,
- Online transactions,
- Internet of Things (IoT),
- Cloud computing,

the amount of data generated every second has increased enormously. This massive collection of data is called **Big Data**.

What is Data?

Data refers to raw facts and figures that can be processed to obtain useful information.

Examples:

- Student marks
- Temperature readings

- Images
- Videos
- Social media posts

When data is processed and organized meaningfully, it becomes **information**.

Types of Data

Data can be classified into three types:

1. Structured Data

Structured data follows a fixed format such as rows and columns in a table.

Examples

- Database tables
- Excel sheets
- Banking records

Characteristics

- Organized
- Easy to store and search
- Stored in relational databases

Example Table

Student_ID	Name	Department	Mark
101	Anu	CSE	85
102	Rahul	ECE	90

2. Unstructured Data

Unstructured data does not follow any fixed format.

Examples

- Images
- Videos
- Emails
- Audio files
- Social media posts

Characteristics

- Difficult to process
- Large in size
- Requires advanced tools

About 80% of organizational data is unstructured.

3. Semi-Structured Data

Semi-structured data has some organizational properties but not a strict table structure.

Examples

- XML
- JSON
- HTML
- Email headers

Example (XML)

```
<student>
  <name>Anu</name>
  <department>CSE</department>
</student>
```

What is Big Data?

Big Data refers to extremely large and complex datasets that cannot be processed efficiently using traditional database systems.

Definition

Big Data is a collection of huge volumes of structured, semi-structured, and unstructured data that grows rapidly and requires advanced technologies for storage, processing, and analysis.

Why Big Data is Important?

Big Data helps organizations:

- Make better decisions
- Predict customer behavior
- Improve business performance
- Detect fraud
- Perform real-time analysis

Real-World Examples of Big Data

Social Media

Facebook, Instagram, and Twitter generate huge amounts of photos, videos, and messages daily.

Healthcare

Hospitals generate patient records, scan images, and monitoring data.

E-Commerce

Amazon and Flipkart analyze customer buying behavior.

Banking

Banks use Big Data for fraud detection and risk analysis.

Transportation

Google Maps analyzes traffic data in real time.

Characteristics of Big Data (5 V's)

Big Data is commonly explained using the 5 V's.

1. Volume

Volume refers to the huge amount of data generated.

Example

Facebook generates terabytes of data daily.

2. Velocity

Velocity refers to the speed at which data is generated and processed.

Example

Live stock market updates.

3. Variety

Variety refers to different types of data.

Examples

- Text
 - Audio
 - Video
 - Images
 - Sensor data
-

4. Veracity

Veracity refers to the accuracy and reliability of data.

Example

Fake or incomplete social media information.

5. Value

Value refers to extracting meaningful insights from data.

Example

Customer recommendation systems.

Need for Big Data

Traditional systems cannot efficiently handle modern data because:

- Data size is extremely large
- Data is generated rapidly
- Data comes in different formats
- Real-time processing is required

Hence Big Data technologies like Hadoop and Spark are used.

Advantages of Big Data

- Better business decisions
- Real-time analytics
- Improved customer experience
- Fraud detection
- Cost reduction
- Improved operational efficiency

Challenges of Big Data

- Storage difficulty
- Data security
- Data privacy

- Processing complexity
- High-speed data generation
- Managing unstructured data

Conventional Data vs Big Data

Conventional Data

Conventional data refers to traditional data that can be stored and processed using conventional database systems such as RDBMS.

Characteristics

- Structured
- Smaller in size
- Stored in tables
- Easy to process

Examples

- Student databases
- Payroll systems
- Banking transaction tables

Big Data

Big Data refers to massive, rapidly growing datasets that traditional systems cannot handle efficiently.

Characteristics

- Huge volume
- High speed
- Multiple formats
- Distributed storage
- Complex processing

Comparison Between Conventional Data and Big Data

Conventional Data	Big Data
Small data size	Extremely large data size
Mostly structured	Structured, semi-structured, unstructured
Stored in centralized databases	Stored in distributed systems
Uses SQL databases	Uses Hadoop, Spark, NoSQL
Easy processing	Complex processing
Low processing speed required	High-speed processing required
Vertical scaling	Horizontal scaling
Traditional analytics	Advanced analytics and machine learning

Traditional BI	Big Data
Centralized server	Distributed file system
Structured data only	All data types
Offline analysis	Real-time + offline analysis
Move data to processing	Move processing to data
Vertical scaling	Horizontal scaling

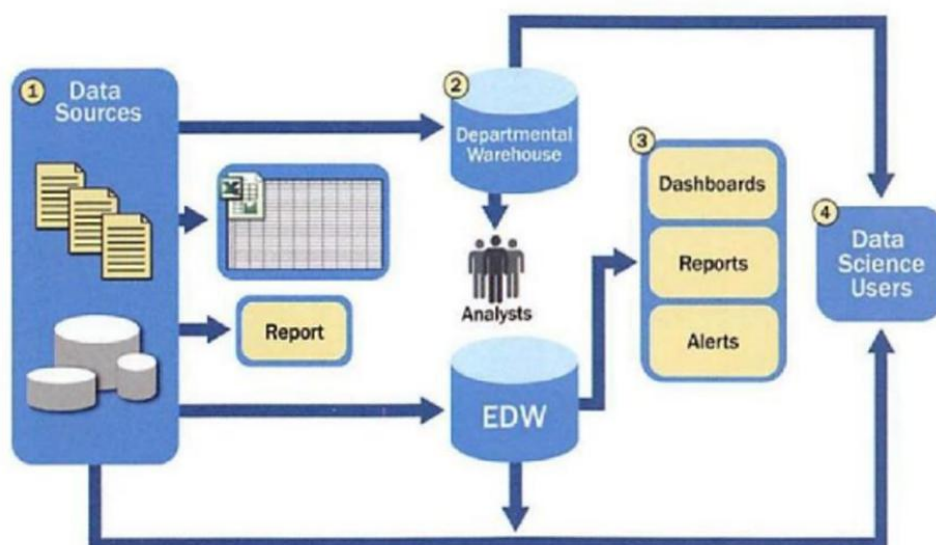
Why Conventional Systems Fail?

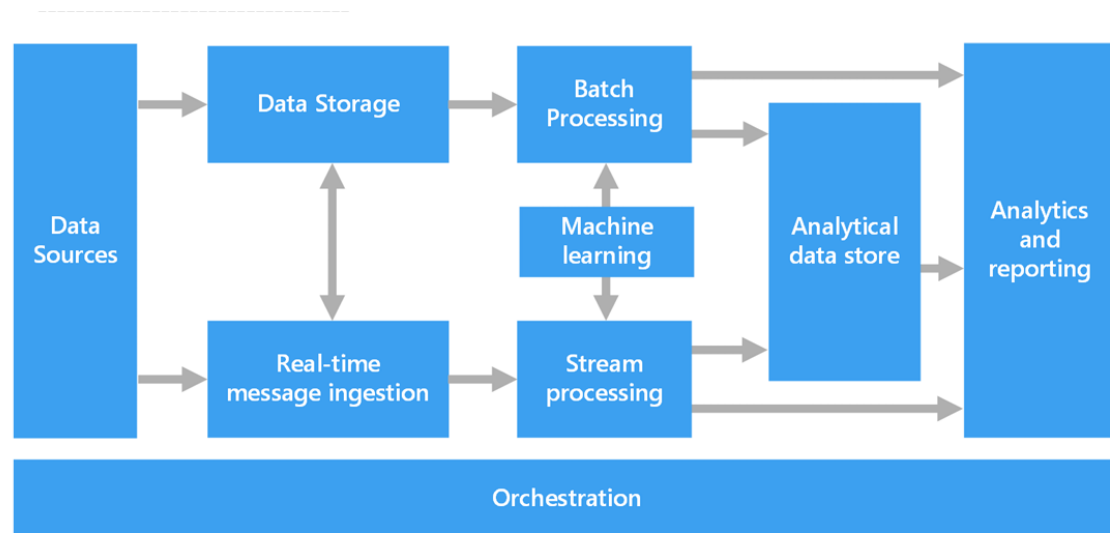
Traditional systems face problems because:

- Storage limitation
- Slow processing
- Cannot handle unstructured data
- High hardware cost
- Poor scalability

Big data architecture

1. For data sources to be loaded into the data warehouse, data needs to be well understood, structured and normalized with the appropriate data type definitions.
2. As a result of this level of control on the EDW(enterprise data warehouse-on server or on cloud), additional local systems may emerge in the form of departmental warehouses and local data marts that business users create to accommodate their need for flexible analysis. However, these local systems reside in isolation, often are not synchronized or integrated with other data stores and may not be backed up.
3. In the data warehouse, data is read by additional applications across the enterprise for BI and reporting purposes.
4. At the end of this workflow, analysts get data from the server. Because users generally are not allowed to run custom or intensive analytics on production databases, analysts create data extracts from the EDW to analyze data offline in R or other local analytical tools to store and process critical data, supporting enterprise applications and enabling corporate reporting activities.





Most big data architectures include some or all of the following components:

- Data sources. All big data solutions start with one or more data sources. Examples include:
 - Application data stores, such as relational databases.
 - Static files produced by applications, such as web server log files.
 - Real-time data sources, such as IoT devices.
- Data storage. Data for batch processing operations is typically stored in a distributed file store that can hold high volumes of large files in various formats. This kind of store is often called a data lake. Options for implementing this storage include Azure Data Lake Store or blob containers in Azure Storage.
- Batch processing. Because the data sets are so large, often a big data solution must process data files using long-running batch jobs to filter, aggregate, and otherwise prepare the data for analysis. Usually these jobs involve reading source files,

processing them, and writing the output to new files. Options include running U-SQL jobs in Azure Data Lake Analytics, using Hive, Pig, or custom Map/Reduce jobs in an HDInsight Hadoop cluster, or using Java, Scala, or Python programs in an HDInsight Spark cluster.

- Real-time message ingestion. If the solution includes real-time sources, the architecture must include a way to capture and store real-time messages for stream processing. This might be a simple data store, where incoming messages are dropped into a folder for processing. However, many solutions need a message ingestion store to act as a buffer for messages, and to support scale-out processing, reliable delivery, and other message queuing semantics. This portion of a streaming architecture is often referred to as stream buffering. Options include Azure Event Hubs, Azure IoT Hub, and Kafka.

- Stream processing. After capturing real-time messages, the solution must process them by filtering, aggregating, and otherwise preparing the data for analysis. The processed stream data is then written to an output sink. Azure Stream Analytics provides a managed stream processing service based on perpetually running SQL queries that operate on unbounded streams. You can also use open source Apache streaming technologies like Storm and Spark Streaming in an HDInsight cluster.

- Analytical data store. Many big data solutions prepare data for analysis and then serve the processed data in a structured format that can be queried using analytical tools. The analytical data store used to serve these queries can be a Kimball-style relational data warehouse, as seen in most traditional business intelligence (BI) solutions. Alternatively, the data could be

presented through a low-latency NoSQL technology such as HBase, or an interactive Hive database that provides a metadata abstraction over data files in the distributed data store. Azure Synapse Analytics provides a managed service for large-scale, cloud-based data warehousing. HDInsight supports Interactive Hive, HBase, and Spark SQL, which can also be used to serve data for analysis.

- **Analysis and reporting.** The goal of most big data solutions is to provide insights into the data through analysis and reporting. To empower users to analyze the data, the architecture may include a data modeling layer, such as a multidimensional OLAP cube or tabular data model in Azure Analysis Services. It might also support self-service BI, using the modeling and visualization technologies in Microsoft Power BI or Microsoft Excel. Analysis and reporting can also take the form of interactive data exploration by data scientists or data analysts. For these scenarios, many Azure services support analytical notebooks, such as Jupyter, enabling these users to leverage their existing skills with Python or R. For large-scale data exploration, you can use Microsoft R Server, either standalone or with Spark.

- **Orchestration.** Most big data solutions consist of repeated data processing operations, encapsulated in workflows, that transform source data, move data between multiple sources and sinks, load the processed data into an analytical data store, or push the results straight to a report or dashboard. To automate these workflows, you can use an orchestration technology such as Azure Data Factory or Apache Oozie and Sqoop

Big data platforms

The rapid growth of data from social media, banking systems, healthcare, sensors, and e-commerce applications created the need for specialized platforms that can store and process huge datasets efficiently.

Traditional database systems cannot handle:

- massive data volume,
- high-speed data generation,
- different data formats,
- and real-time analytics.

Therefore, Big Data Platforms are used.

A Big Data Platform is an integrated system used for storing, processing, analyzing, and managing Big Data.

Definition

A Big Data Platform is an enterprise-level IT solution that combines storage systems, processing frameworks, analytics tools, and management software to handle Big Data efficiently.

It provides a complete environment for:

- data storage,
- distributed processing,
- analytics,
- visualization,

- and reporting.

Features of Big Data Platforms

A good Big Data Platform should have the following features:

1. Scalability

The platform should support increasing data volume by adding more systems or nodes.

Big Data Platforms mainly use **horizontal scaling**.

2. Distributed Storage

Data is stored across multiple computers instead of a single machine.

This improves:

- reliability,
 - availability,
 - and performance.
-

3. High-Speed Processing

Big Data Platforms process large datasets quickly using parallel processing techniques.

4. Fault Tolerance

If one system fails, data should not be lost.

Example:

Hadoop stores multiple copies of data blocks.

5. Support for Different Data Types

The platform should support:

- Structured data
 - Semi-structured data
 - Unstructured data
-

6. Real-Time Processing

Modern applications require real-time analytics.

Examples:

- Fraud detection
 - Traffic monitoring
 - Social media analytics
-

7. Analytics and Reporting

Big Data Platforms provide tools for:

- Data analysis
- Visualization
- Reporting
- Machine learning

Components of Big Data Platform

A Big Data Platform generally contains the following components:

Component	Function
Data Sources	Generate data
Storage Layer	Stores data
Processing Engine	Processes data
Analytics Tools	Analyze data
Visualization Tools	Create reports and dashboards

Types of Big Data Platforms

Big Data Platforms are mainly classified into:

1. Open-source platforms
2. Commercial platforms
3. Cloud-based platforms

Some of the popular Big Data Platforms are:

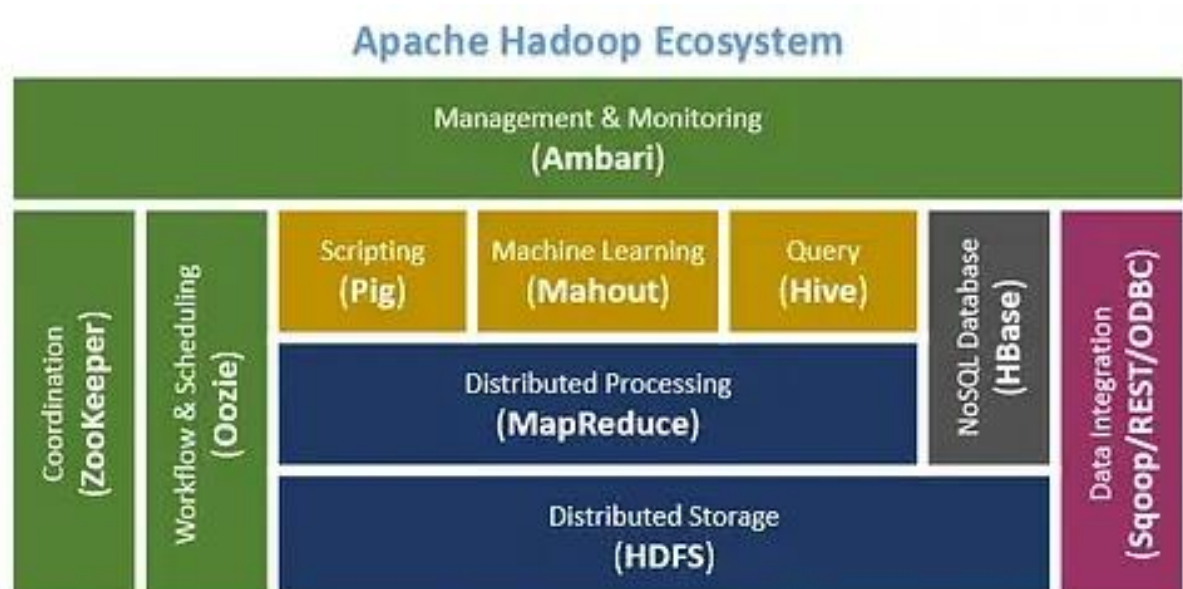
1. Apache Hadoop
2. Apache Spark
3. Amazon Web Services (AWS)

4. Cloudera
5. Hortonworks
6. MapR
7. Microsoft HDInsight
8. IBM Open Platform

1. Apache Hadoop

Apache Hadoop is the most popular open-source Big Data Platform. It is designed for distributed storage and parallel processing of massive datasets using clusters of commodity computers.

Hadoop was developed under the Apache Software Foundation.



Features of Hadoop

- Open-source platform
- Distributed storage using HDFS
- Parallel processing using MapReduce
- Fault tolerant
- Highly scalable
- Cost-effective

Core Components of Hadoop

1. HDFS (Hadoop Distributed File System)

Used for distributed storage of large datasets.

2. MapReduce

Used for parallel data processing.

3. YARN

Used for resource management and job scheduling.

Hadoop Ecosystem Tools

Tool	Purpose
Hive	SQL-like querying
Pig	Data flow scripting
HBase	NoSQL database

Tool Purpose

Sqoop Data transfer

Flume Log collection

Spark Fast data processing

Advantages of Hadoop

- Can handle petabytes of data
 - Reliable and fault tolerant
 - Supports distributed processing
 - Works on low-cost hardware
-

Limitations of Hadoop

- Complex ecosystem
- Slower than Spark for iterative tasks
- Requires skilled administrators

2. Apache Spark

Apache Spark is a fast Big Data processing framework developed for high-speed analytics.

It performs in-memory processing, making it much faster than Hadoop MapReduce.

Features of Apache Spark

- In-memory processing
 - High-speed computation
 - Supports real-time streaming
 - Supports machine learning
 - Easy integration with Hadoop
-

Components of Spark

Component	Function
Spark Core	Basic processing engine
Spark SQL	Structured data processing
Spark Streaming	Real-time data processing
MLlib	Machine learning library
GraphX	Graph processing

Advantages of Spark

- Faster than MapReduce
 - Supports batch and streaming analytics
 - Easy programming support
-

Applications of Spark

- Real-time analytics

- Fraud detection
- Machine learning
- Recommendation systems

3. Amazon Web Services (AWS)

AWS is a cloud-based Big Data Platform provided by Amazon. AWS provides scalable cloud infrastructure for Big Data storage and analytics.

Amazon EMR (Elastic MapReduce)

Amazon EMR is a cloud service used to:

- Create Hadoop clusters
 - Run Spark applications
 - Process Big Data in cloud
-

Features of AWS

- Cloud-based infrastructure
 - On-demand scalability
 - Supports Hadoop and Spark
 - Pay-as-you-use model
-

Advantages of AWS

- No hardware maintenance

- Easy deployment
 - Highly scalable
 - Cost-effective
-

Applications of AWS

- Cloud analytics
 - Data warehousing
 - Real-time processing
-

4. Cloudera

Cloudera is a commercial Hadoop-based Big Data Platform.

It provides enterprise-level Big Data solutions for analytics and data management.

Features of Cloudera

- Cluster management
 - Real-time analytics
 - Security and governance
 - Data engineering support
-

Products of Cloudera

- Cloudera Data Platform (CDP)
- Cloudera Data Science Workbench

- Cloudera Operational Database
-

Advantages of Cloudera

- Enterprise support
 - Easy cluster monitoring
 - Strong security features
-

5. Hortonworks

Hortonworks is another Hadoop-based Big Data Platform built completely on open-source technologies.

Features of Hortonworks

- 100% open-source
 - Cluster management tools
 - Security and governance
 - Distributed data processing
-

Advantages of Hortonworks

- Enterprise-ready Hadoop distribution
 - Centralized administration
 - Open-source ecosystem
-

6. MapR

MapR is a distributed Big Data Platform used for large-scale analytics.

Unlike Hadoop, MapR uses a Unix-based file system instead of HDFS.

Features of MapR

- High-performance storage
- Real-time processing
- Hadoop and Spark integration
- Supports distributed applications

Advantages of MapR

- Faster performance
- Easy file access
- Reliable architecture

7. Microsoft HDInsight

HDInsight is Microsoft Azure's cloud-based Hadoop service. It provides Hadoop clusters on the Azure cloud platform.

Features of HDInsight

- Cloud-based Hadoop platform
- Supports Spark, Hive, Kafka, and HBase

- Easy integration with Azure services
-

Advantages of HDInsight

- Easy cloud deployment
 - Scalable infrastructure
 - Microsoft ecosystem integration
-

8. IBM Open Platform

IBM Open Platform is IBM's Hadoop-based Big Data solution. It supports analytics, machine learning, and distributed processing.

Features of IBM Open Platform

- Hadoop ecosystem support
 - Spark integration
 - Cluster monitoring
 - Enterprise analytics tools
-

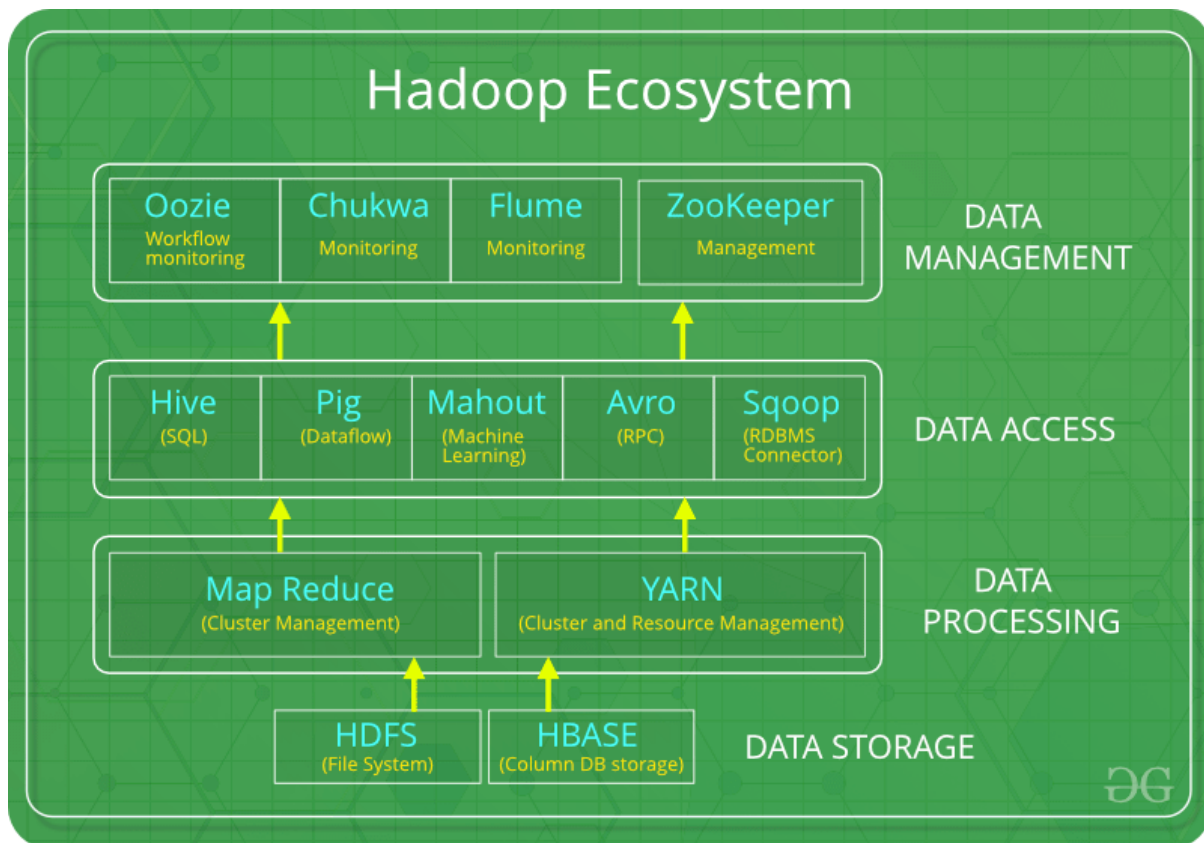
Advantages of IBM Open Platform

- Enterprise-grade security
 - Strong analytics support
 - Reliable performance
-

Comparison of Popular Big Data Platforms

Platform	Type	Major Feature
Hadoop	Open-source	Distributed storage
Spark	Open-source	Fast in-memory processing
AWS	Cloud-based	Scalable cloud analytics
Cloudera	Commercial	Enterprise Hadoop
Hortonworks	Open-source	Hadoop distribution
MapR	Commercial	High-performance storage
HDInsight	Cloud-based	Azure Hadoop service
IBM Open Platform	Commercial	Enterprise analytics

Hadoop Ecosystem



HDFS

- HDFS is the primary or major component of Hadoop ecosystem and is responsible for storing large data sets of structured or unstructured data across various nodes and thereby maintaining the metadata in the form of log files.
- HDFS maintains all the coordination between the clusters and hardware, thus working at the heart of the system.
- Name Node is the prime node which contains metadata (data about data) requiring comparatively fewer resources than the data nodes that stores the actual data.

YARN

YARN is the one who helps to manage the resources across the clusters. In short, it performs scheduling and resource

allocation for the Hadoop System. Consists of three major components i.e.

1. Resource Manager
2. Nodes Manager
3. Application Manager

- Resource manager has the privilege of allocating resources for the applications in a system whereas Node managers work on the allocation of resources such as CPU, memory, bandwidth per machine and later on acknowledges the resource manager. Application manager works as an interface between the resource manager and node manager and performs negotiations as per the requirement of the two.

MapReduce:

- By making the use of distributed and parallel algorithms, MapReduce makes it possible to carry over the processing's logic and helps to write applications which transform big data sets into a manageable one.
- Map() performs sorting and filtering of data and thereby organizing them in the form of group. Map generates a key-value pair based result which is later on processed by the Reduce() method
- Reduce(), as the name suggests does the summarization by aggregating the mapped data. In simple, Reduce() takes the output generated by Map() as input and combines those tuples into smaller set of tuples.

PIG

Pig was basically developed by Yahoo which works on a pig Latin language, which is Query based language similar to SQL.

- It is a platform for structuring the data flow, processing and analyzing huge data sets.
- Pig does the work of executing commands and in the background, all the activities of MapReduce are taken care of. After the processing, pig stores the result in HDFS.

HIVE

- With the help of SQL methodology and interface, HIVE performs reading and writing of large data sets. However, its query language is called as HQL (Hive Query Language).
- It is highly scalable as it allows real-time processing and batch processing both. Also, all the SQL datatypes are supported by Hive thus, making the query processing easier.
- Similar to the Query Processing frameworks, HIVE too comes with two components: JDBC Drivers and HIVE Command Line.
- JDBC, along with ODBC drivers work on establishing the data storage permissions and connection whereas HIVE Command line helps in the processing of queries.

Mahout:

- Mahout, allows Machine Learnability to a system or application. Machine Learning, as the name suggests helps the system to develop itself based on some patterns, user/environmental interaction or on the basis of algorithms.

- It provides various libraries or functionalities such as collaborative filtering, clustering, and classification which are nothing but concepts of Machine learning. It allows invoking algorithms as per our need with the help of its own libraries.

Apache Spark:

- It's a platform that handles all the process consumptive tasks like batch processing, interactive or iterative real-time processing, graph conversions, and visualization, etc.
- It consumes in memory resources hence, thus being faster than the prior in terms of optimization.
- Spark is best suited for real-time data whereas Hadoop is best suited for structured data or batch processing, hence both are used in most of the companies interchangeably.

OOZIE

- Apache Oozie is a workflow scheduler for Hadoop. It is a system which runs the workflow of dependent jobs.
- Here, users are permitted to create Directed Acyclic Graphs of workflows, which can be run in parallel and sequentially in Hadoop.

It consists of two parts:

- Workflow engine: Responsibility of a workflow engine is to store and run workflows composed of Hadoop jobs e.g., MapReduce, Pig, Hive.
- Coordinator engine: It runs workflow jobs based on predefined schedules and availability of data.

Sqoop in Hadoop

- Sqoop is a powerful tool designed to transfer data between Hadoop and relational database servers.
- It is particularly useful for importing data from relational databases such as MySQL, Oracle to Hadoop HDFS, and exporting data from Hadoop file system to relational databases

Avro

- Apache Avro is a language-neutral data serialization system.
- It was developed by Doug Cutting, the father of Hadoop. Since Hadoop writable classes lack language portability, Avro becomes quite helpful, as it deals with data formats that can be processed by multiple languages.
- Avro is a preferred tool to serialize data in Hadoop.

ZooKeeper in Hadoop

- ZooKeeper is a distributed, open-source coordination service designed to manage and coordinate large sets of hosts in a distributed environment.
- It provides a simple set of primitives to implement higher-level services for synchronization, configuration maintenance, and group and naming.
- ZooKeeper is widely used in distributed systems such as Hadoop, Kafka, and HBase, and it has become an essential component of many distributed applications.

Apache Flume

- Apache Flume is a reliable and distributed system for collecting, aggregating and moving massive quantities of log data.

- It has a simple yet flexible architecture based on streaming data flows.
- Apache Flume is used to collect log data present in log files from web servers and aggregating it into HDFS for analysis

Chukwa

- Chukwa is a Hadoop subproject devoted to large-scale log collection and analysis.
- Chukwa is built on top of the Hadoop distributed filesystem (HDFS) and MapReduce framework and inherits Hadoop's scalability and robustness.
- Chukwa also includes a flexible and powerful toolkit for displaying monitoring and analyzing results, in order to make the best use of this collected data.

Apache HBase:

- It's a NoSQL database which supports all kinds of data and thus capable of handling anything of Hadoop Database. It provides capabilities of Google's BigTable, thus able to work on Big Data sets effectively.
- At times where we need to search or retrieve the occurrences of something small in a huge database, the request must be processed within a short quick span of time. At such times, HBase comes handy as it gives us a tolerant way of storing limited data.

HDFS Architecture

1. Role of HDFS in Hadoop Architecture
2. What is HDFS in Hadoop
3. How HDFS Works

- HDFS(Hadoop Distributed File System) is utilized for storage permission in a Hadoop cluster.
- It is used to scale a single Apache Hadoop cluster to hundreds (and even thousands) of nodes.
- HDFS is designed in such a way that ,more in storing the data in a large chunk of blocks rather than storing small data blocks.
- HDFS in Hadoop provides Fault-tolerance and High availability to the storage layer and the other devices present in that Hadoop cluster.

Goals of HDFS

Fast recovery from hardware failures

- One HDFS instance may consist of thousands of servers, failure of at least one server is inevitable.
- HDFS has been built to detect faults and automatically recover quickly.

Access to streaming data

- HDFS is intended more for batch processing, so applications are designed for high data throughput rates.

Accommodation of large data sets

- HDFS accommodates applications that have data sets typically gigabytes to terabytes in size.
- HDFS provides high aggregate data bandwidth and can scale to hundreds of nodes in a single cluster.

Portability

- HDFS is designed to be portable across multiple hardware platforms and to be compatible with a variety of underlying operating systems.

<..\Big data\Hadoop-M3 full.pdf>

Introduction to Data Lakes and Data Lakehouse Architecture

Introduction

Modern organizations generate massive amounts of:

- structured data,
- semi-structured data,
- and unstructured data.

Traditional databases and data warehouses are not flexible enough to efficiently handle all these data types at very large scale.

To solve this problem, modern Big Data systems use:

1. Data Lakes
2. Data Lakehouse Architecture

These architectures provide scalable storage, faster analytics, and support for AI and machine learning applications.

4.2 What is a Data Lake?

A **Data Lake** is a centralized storage repository that stores huge amounts of raw data in its original format.

The data may be:

- structured,
- semi-structured,
- or unstructured.

Unlike traditional databases, Data Lakes do not require data to be organized before storing.

Definition

A Data Lake is a large storage system that stores raw data from multiple sources in its native format until it is needed for analysis.

4.3 Characteristics of Data Lake

1. Stores All Types of Data

A Data Lake can store:

- tables,
- images,

- videos,
 - logs,
 - sensor data,
 - emails,
 - JSON,
 - XML files.
-

2. Schema-on-Read

In Data Lakes, data structure is applied only when data is read. This is called **Schema-on-Read**.

This provides high flexibility.

3. Distributed Storage

Data is stored across distributed systems such as:

- Hadoop HDFS,
 - Amazon S3,
 - Azure Data Lake Storage.
-

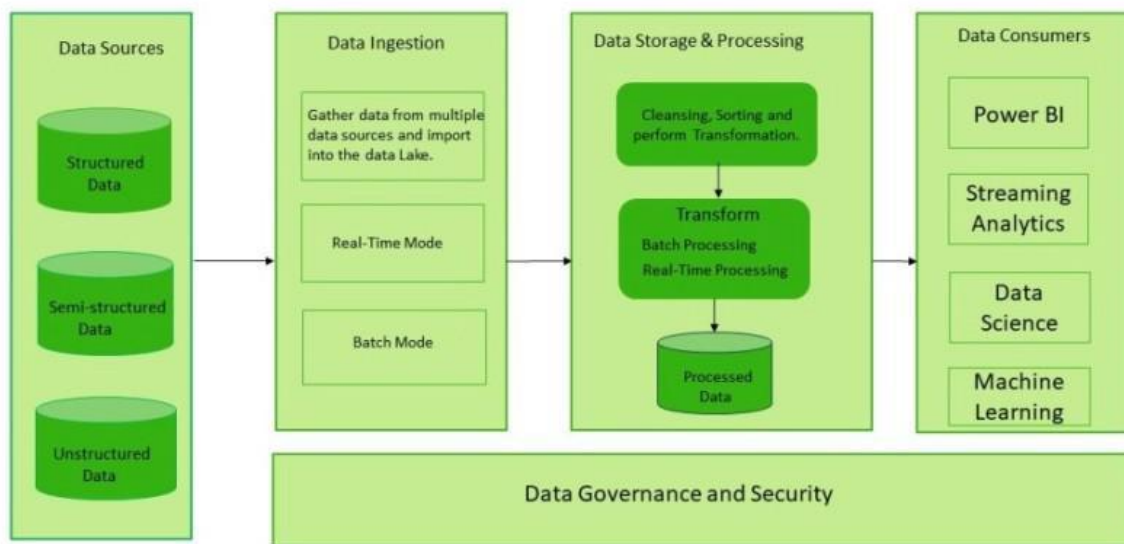
4. Scalable

Storage can be increased easily by adding more nodes or cloud storage.

5. Low-Cost Storage

Data Lakes use commodity hardware or cloud storage, making them cost-effective.

4.4 Architecture of Data Lake



A Data Lake architecture is designed to collect, store, process, and analyze huge amounts of data from multiple sources. It supports:

- Structured data
- Semi-structured data
- Unstructured data

The architecture also supports both:

- Real-time processing
- Batch processing

The main components of Data Lake architecture are shown below.

Data Lake Architecture

Data Lake Architecture

Components of Data Lake Architecture

1. Data Sources

Data is collected from different sources in various formats.

Types of Data Sources

a) Structured Data

Data stored in rows and columns.

Examples:

- Banking records
 - Student databases
 - Sales reports
-

b) Semi-Structured Data

Data with partial organization.

Examples:

- XML
 - JSON
 - Emails
-

c) Unstructured Data

Data without fixed format.

Examples:

- Images
 - Videos
 - Audio files
 - Social media posts
-

2. Data Ingestion

The Data Ingestion layer gathers data from multiple sources and imports it into the Data Lake.

It supports two modes:

a) Real-Time Mode

Data is processed immediately after generation.

Examples:

- Sensor data
 - Online transactions
 - Live social media feeds
-

b) Batch Mode

Large volumes of data are collected and processed periodically.

Examples:

- Daily reports
 - Monthly transaction records
-

3. Data Storage and Processing

This layer stores and processes the collected data.

Main Functions

a) Cleansing

Removes:

- duplicate data,
 - incorrect data,
 - unnecessary data.
-

b) Sorting

Organizes data into proper order.

c) Transformation

Converts raw data into useful format for analysis.

Processing can be:

- Batch Processing
- Real-Time Processing

The processed data is then stored for analytics.

4. Data Consumers

The processed data is used by different applications and users.

Main Data Consumers

a) Power BI

Used for:

- reports,
 - dashboards,
 - visualization.
-

b) Streaming Analytics

Used for analyzing real-time data streams.

c) Data Science

Used for:

- statistical analysis,
 - predictive analytics,
 - research.
-

d) Machine Learning

Used for:

- AI applications,
 - recommendation systems,
 - prediction models.
-

5. Data Governance and Security

This layer provides:

- data security,

- access control,
- monitoring,
- data quality management.

It ensures safe and authorized access to data.

Working of Data Lake Architecture

1. Data is collected from different sources.
 2. Data is imported through the ingestion layer.
 3. Data is stored in the Data Lake.
 4. Data is cleaned, sorted, and transformed.
 5. Processed data is analyzed using analytics and machine learning tools.
 6. Governance and security mechanisms protect the data.
-

4.5 Advantages of Data Lake

1. Supports all types of data
 2. Highly scalable
 3. Low-cost storage
 4. Supports machine learning and AI
 5. Suitable for Big Data analytics
 6. Supports real-time and batch processing
-

4.6 Limitations of Data Lake

1. Difficult data management
 2. Security challenges
 3. Data quality issues
 4. Poorly managed lakes become “Data Swamps”
-

Data Swamp

A poorly managed Data Lake is called a **Data Swamp**.

In a Data Swamp:

- data becomes disorganized,
 - duplicated,
 - and difficult to analyze.
-

4.7 What is Data Warehouse?

A Data Warehouse stores processed and structured data for reporting and business analysis.

Characteristics

- Structured data only
 - Schema-on-write
 - High query performance
 - Used for reporting and BI
-

4.8 Difference Between Data Lake and Data Warehouse

Data Lake

Stores raw data

Supports all data types

Schema-on-read

Used for AI and ML

Low-cost storage

Data Warehouse

Stores processed data

Mainly structured data

Schema-on-write

Used for reporting

Expensive storage

4.9 What is Data Lakehouse Architecture?

A Data Lakehouse combines the features of:

- Data Lake and
- Data Warehouse

It provides:

- flexible storage,
- strong data management,
- and fast analytics.

4.10 Features of Data Lakehouse

1. Unified storage system
2. Supports all data types
3. ACID transaction support
4. Better data governance

5. High-performance analytics
 6. Supports AI and machine learning
-

4.11 Advantages of Data Lakehouse

1. Combines benefits of Data Lake and Warehouse
 2. Better security and governance
 3. Faster analytics
 4. Supports machine learning
 5. Reduces data duplication
-

4.12 Applications

Data Lake Applications

- Social media analytics
 - IoT data storage
 - Video analytics
 - Log analysis
-

Data Lakehouse Applications

- Business intelligence
- AI and machine learning
- Real-time analytics
- Enterprise data platforms

AWS EMR (Elastic MapReduce)

AWS (Amazon Web Services) is a cloud computing platform provided by Amazon.

It provides:

- storage,
- computing power,
- databases,
- networking,
- analytics,
- and Big Data services through the cloud.

AWS allows organizations to process huge datasets without maintaining physical hardware.

5.2 What is AWS EMR?

AWS EMR stands for **Amazon Elastic MapReduce**.

It is a cloud-based Big Data platform used for:

- processing large datasets,
- running Hadoop and Spark applications,
- and performing Big Data analytics.

EMR simplifies the setup and management of Hadoop clusters in the cloud.

Definition

Amazon EMR is a managed cluster platform that uses Hadoop, Spark, and other Big Data frameworks to process and analyze massive amounts of data.

5.3 Features of AWS EMR

1. Cloud-based Hadoop platform
 2. Easy cluster setup
 3. Supports Hadoop, Spark, Hive, and HBase
 4. Automatic scaling
 5. Cost-effective
 6. Fast data processing
 7. Integration with Amazon S3
-

5.4 Advantages of AWS EMR

1. No need to maintain hardware
 2. Easy deployment of Hadoop clusters
 3. Highly scalable
 4. Supports distributed processing
 5. Pay-as-you-use pricing
 6. Faster processing of Big Data
-

5.5 Components Used in AWS EMR

Component	Purpose
EMR Cluster	Group of EC2 machines for processing
Hadoop	Distributed processing framework
MapReduce	Parallel data processing
Amazon S3	Cloud storage
EC2 Instances	Virtual servers

5.6 What is an EMR Cluster?

An EMR Cluster is a collection of Amazon EC2 instances used to run Big Data frameworks such as:

- Hadoop,
- Spark,
- Hive,
- and HBase.

The cluster performs distributed processing of large datasets.

Types of Nodes in EMR Cluster

1. Master Node

- Manages the cluster
- Coordinates processing tasks

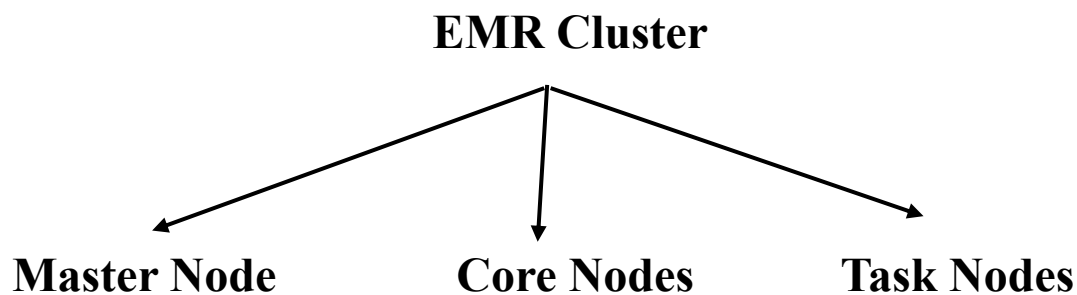
2. Core Node

- Stores data
 - Processes tasks
-

3. Task Node

- Performs processing tasks only
 - Does not store data
-

Diagram: AWS EMR Cluster



5.7 Steps for EMR Cluster Setup

The following steps are used to create an EMR cluster in AWS.

Step 1: Login to AWS Console

Open:

- AWS Management Console
 - Select EMR service
-

Step 2: Create Cluster

Choose:

- Create Cluster
 - Configure cluster settings
-

Step 3: Select Software

Choose required Big Data tools:

- Hadoop
 - Spark
 - Hive
 - HBase
-

Step 4: Configure Hardware

Select:

- Number of instances
 - Instance type
 - Storage capacity
-

Step 5: Configure Security

Configure:

- Access permissions
 - Key pairs
 - IAM roles
-

Step 6: Launch Cluster

Click:

- Create Cluster

AWS automatically creates the Hadoop cluster.

5.8 Amazon S3 in EMR

Amazon S3 (Simple Storage Service) is cloud storage provided by AWS.

EMR uses S3 to:

- store input data,
 - store output data,
 - and store Big Data files.
-

Advantages of Using S3

1. Highly scalable
 2. Reliable storage
 3. Low-cost cloud storage
 4. Easy integration with EMR
-

5.9 Running MapReduce Jobs on S3

In AWS EMR, MapReduce jobs can directly read data from Amazon S3 and write results back to S3.

This eliminates the need for local storage.

Steps to Run MapReduce Jobs on S3

Step 1: Upload Data to S3

Store input files in an S3 bucket.

Example:

`s3://mybucket/input/`

Step 2: Create EMR Cluster

Launch Hadoop cluster using AWS EMR.

Step 3: Submit MapReduce Job

Run the MapReduce program on the cluster.

The program reads input data from S3.

Step 4: Process Data

Hadoop MapReduce processes data in parallel across cluster nodes.

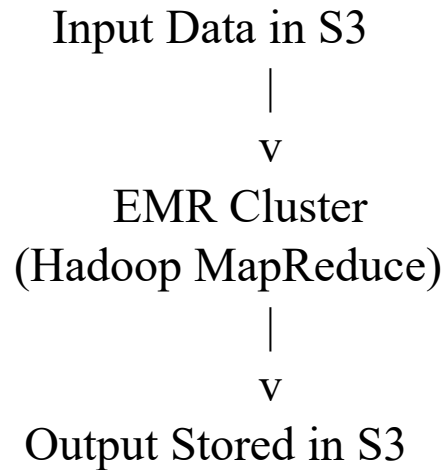
Step 5: Store Output in S3

The processed output is stored back into S3.

Example:

`s3://mybucket/output/`

Diagram: Running MapReduce Job on S3



5.10 Advantages of Running MapReduce on S3

1. No local storage required
2. Easy scalability
3. Faster distributed processing
4. Reliable cloud storage
5. Cost-effective solution

5.11 Applications of AWS EMR

Area	Application
Banking	Fraud detection
Healthcare	Medical data analysis
Social Media	Sentiment analysis
E-commerce	Recommendation systems

Area	Application
IoT	Sensor data processing

Databricks

Introduction to Databricks

Databricks is a cloud-based Unified Analytics Platform built on top of Apache Spark. It provides a collaborative environment for data engineering, data analytics, data science, and machine learning.

Databricks was developed by the creators of Apache Spark to simplify Big Data processing and analytics.

It combines:

- Data Engineering
- Data Analytics
- Data Science
- Machine Learning
- Data Visualization

into a single platform.

What is a Unified Analytics Platform?

Traditionally, organizations used separate tools for:

- Data storage
- Data processing

- Data analysis
- Machine learning
- Visualization

This resulted in:

- data duplication,
- increased complexity,
- and higher maintenance costs.

A **Unified Analytics Platform** integrates all these functions into a single environment.

Definition

A Unified Analytics Platform is an integrated system that enables data engineers, analysts, and data scientists to work together on the same data using a common platform.

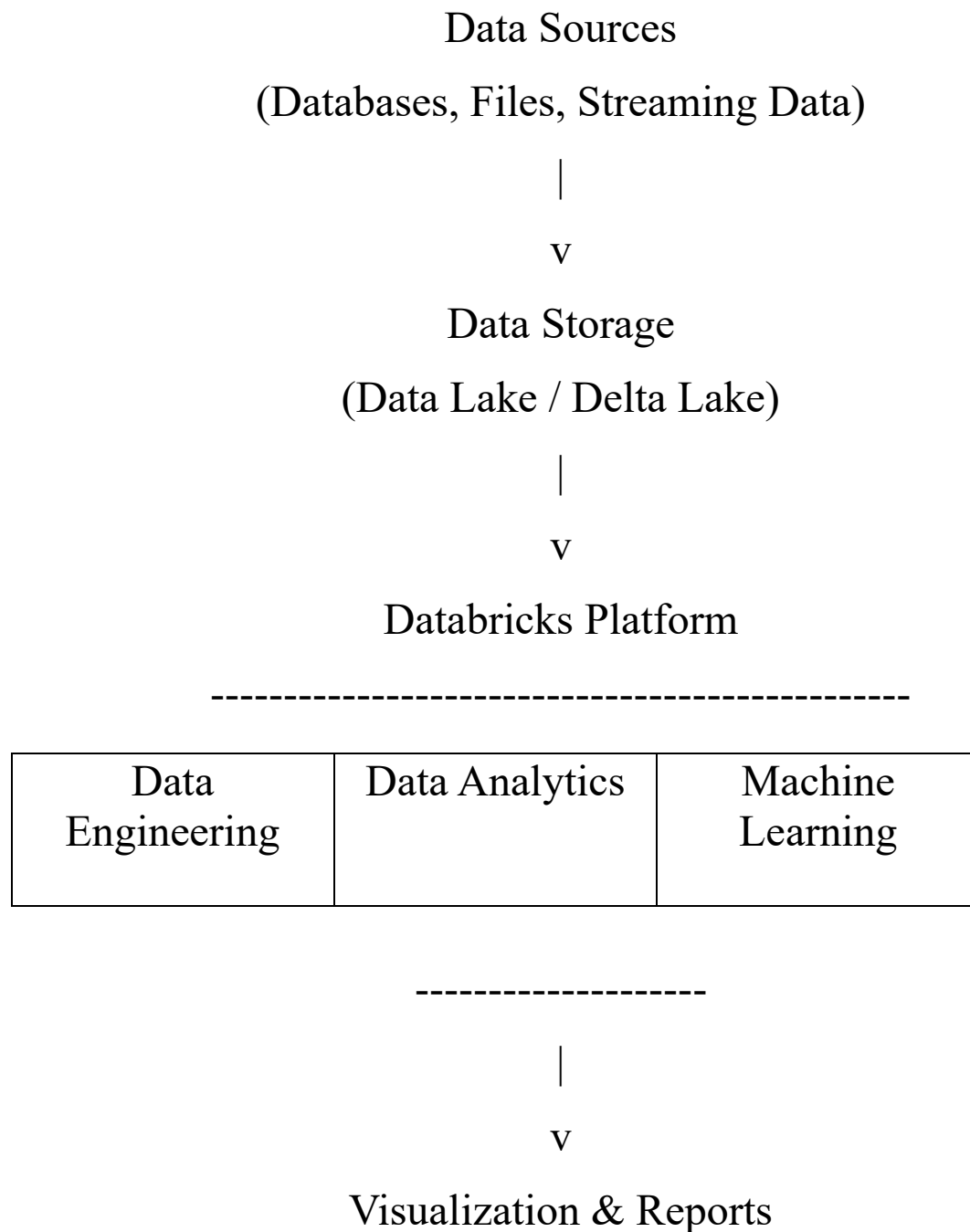
Need for Databricks

Organizations generate huge amounts of data and require a platform that can:

- Store large datasets
- Process data quickly
- Support real-time analytics
- Build machine learning models
- Allow team collaboration

Databricks addresses these requirements through a unified cloud-based platform.

Architecture of Databricks Unified Analytics Platform



Components of Databricks Unified Analytics Platform

1. Data Sources

Data can be collected from:

- Databases
 - CSV/JSON files
 - Streaming data
 - Cloud storage
-

2. Data Storage Layer

Databricks stores data in:

- Data Lakes
- Delta Lake
- Cloud Storage

This layer stores both raw and processed data.

3. Data Engineering

Used for:

- Data ingestion
- ETL operations
- Data transformation
- Building data pipelines

Apache Spark is used for large-scale data processing.

4. Data Analytics

Used for:

- SQL queries

- Data analysis
 - Dashboard creation
 - Business reporting
-

5. Machine Learning

Supports:

- Model building
- Model training
- Prediction
- AI applications

Databricks integrates with ML libraries such as MLlib.

6. Visualization and Reporting

Results can be displayed as:

- Charts
- Dashboards
- Reports

for business decision making.

Advantages of Databricks Unified Analytics Platform

1. Single platform for analytics and machine learning
2. Built on Apache Spark
3. Supports team collaboration

4. Scalable cloud infrastructure
5. Faster Big Data processing

Databricks Notebooks

Introduction

A **Databricks Notebook** is an interactive workspace used for writing, executing, and sharing code.

It allows users to perform:

- Data analysis
- Data processing
- Machine learning
- Visualization

within a single environment.

Databricks Notebooks support multiple programming languages including:

- Python
- SQL
- Scala
- R

Features of Databricks Notebooks

1. Multi-language Support

Users can write code in Python, SQL, Scala, and R within the same notebook.

2. Interactive Execution

Code can be executed cell by cell, making debugging and analysis easier.

3. Collaboration

Multiple users can work on the same notebook simultaneously.

4. Visualization Support

Results can be displayed as:

- Tables
- Charts
- Graphs
- Dashboards

5. Integration with Spark

Notebooks can directly access Spark clusters for distributed data processing.

Advantages of Databricks Notebooks

- Easy code development
 - Real-time collaboration
 - Interactive data analysis
 - Integrated visualization
 - Supports machine learning workflows
-

Delta Engine Overview

Introduction

The **Delta Engine** is a high-performance query engine used in Databricks to improve the speed and reliability of data processing.

It is built on top of **Delta Lake** and Apache Spark.

The Delta Engine optimizes:

- SQL queries
 - Data analytics
 - Machine learning workloads
-

Features of Delta Engine

1. Faster Query Execution

Optimizes Spark query processing for better performance.

2. ACID Transactions

Ensures reliable and consistent data operations.

3. Efficient Data Storage

Stores data in optimized formats for faster access.

4. Scalability

Handles large-scale Big Data workloads efficiently.

5. Integration with Delta Lake

Provides reliable storage and analytics capabilities.

Benefits of Delta Engine

- Improved performance
 - Reliable data management
 - Faster analytics
 - Better scalability
-

On-Premise vs Cloud Big Data Deployment Tradeoffs

Organizations can deploy Big Data platforms either:

1. On-Premise
2. Cloud-Based

Each approach has its own advantages and limitations.

On-Premise Big Data Deployment

In on-premise deployment, all hardware and software are installed and managed within the organization's own data center.

Advantages

- Complete control over infrastructure
- Better customization
- Higher data privacy

Limitations

- High initial cost
- Maintenance responsibility

- Limited scalability
-

Cloud Big Data Deployment

In cloud deployment, Big Data infrastructure is provided through cloud platforms such as:

- AWS
- Microsoft Azure
- Google Cloud

Advantages

- Easy scalability
- Lower initial investment
- Pay-as-you-use pricing
- Reduced maintenance effort

Limitations

- Dependence on internet connectivity
 - Data security concerns
 - Less direct control over infrastructure
-

Comparison: On-Premise vs Cloud Deployment

Feature	On-Premise	Cloud-Based
Infrastructure Ownership	Organization	Cloud Provider
Initial Cost	High	Low
Scalability	Limited	Highly Scalable
Maintenance	Managed by Organization	Managed by Provider
Deployment Time	Longer	Faster
Flexibility	Less Flexible	Highly Flexible
Security Control	Higher	Shared Responsibility